

KERNEL-LEVEL DEFENSE

eBPF Sensor Engineering
& Rootkit Analysis.

Disusun Oleh: **Rizki Angga Pratama**
L2 SOC Analyst

HAK CIPTA & DISCLAIMER

Kernel-Level Defense: eBPF Sensor Engineering & Rootkit Analysis
Edisi Enterprise (50 Halaman)

Disusun oleh: Rizki Angga Pratama, S. Kom, C. Pro

Buku ini dirancang bagi Security Engineers, Threat Hunters, dan System Administrators yang ingin memahami pertahanan di lapisan paling dalam sistem operasi: Kernel (Ring 0). Mengamankan aplikasi di level pengguna tidak ada artinya jika kernel yang mendasari sistem tersebut telah dikompromikan oleh *Rootkit* siluman.

> **Revolusi eBPF:**
> Secara historis, memodifikasi kernel untuk menambahkan sensor keamanan membutuhkan penulisan *Kernel Modules* (C/C++) yang rawan menyebabkan *Kernel Panic (Crash)*. Dengan eBPF (*Extended Berkeley Packet Filter*), kita kini dapat mengeksekusi program di dalam kotak pasir (*Sandboxed VM*) kernel yang dijamin aman dan sangat cepat.

Filosofi Desain:
Disajikan dengan tema *Void Black & Bumblebee Yellow*. Melambangkan gelapnya jurang sistem operasi dan sengatan cahaya teknologi sensor peringatan dini yang meneranginya.

© 2026 Hak cipta dilindungi undang-undang. Panduan Rekayasa Sensor Kernel.

KATA PENGANTAR

Sebagian besar produk EDR (*Endpoint Detection and Response*) beroperasi dengan menanamkan kaitan (*Hooks*) di ruang pengguna (*User-Space*) atau memodifikasi panggilan fungsi `libc`. Namun, peretas tingkat mahir (APT) telah lama menyadari bahwa mereka bisa mem-*bypass* seluruh pengawasan ini dengan turun satu tingkat ke bawah, langsung mengendalikan sang wasit itu sendiri: **Kernel Linux**.

Melalui modul *Rootkit* biner (LKM), penyerang mencegat struktur internal sistem. Ketika Administrator mengetik `ps -ef` untuk melihat proses yang berjalan, perintah tersebut memanggil fungsi kernel. *Rootkit* yang telah bersarang di kernel akan memanipulasi balasan fungsi tersebut, menghapus secara ajaib *Malware* miliknya dari daftar kembalian. Di layar terminal Administrator, sistem tampak bersih sempurna, padahal mesin tersebut sedang digunakan untuk menambang kripto atau mengeksfiltrasi data.

Bagaimana Anda mendeteksi musuh jika Anda tidak bisa mempercayai mata Anda sendiri (Sistem Operasi)? Jawabannya adalah teknologi terhebat dekade ini: **eBPF (Extended BPF)**. eBPF mengizinkan Insinyur Keamanan memprogram sensor mikro *Event-Driven* yang ditanam di jantung kernel tanpa mengubah *Source Code* kernel. Buku ini adalah panduan membongkar ilusi *Rootkit* menggunakan cahaya eBPF.

— Rizki Angga Pratama

Author

DAFTAR ISI (1/2)

BAGIAN I: Anatomi Kernel Linux & Rootkits

Bab 1.1 Model Hak Istimewa (Ring 0 vs Ring 3) Hlm. 7

Bab 1.2 Loadable Kernel Modules (LKM) Hlm. 8

Bab 1.3 System Call Table (sys_call_table) Hooking Hlm. 9

Bab 1.4 Direct Kernel Object Manipulation (DKOM) Hlm. 10

Bab 1.5 VFS & Memanipulasi Daftar File (Hidden Files) Hlm. 11

Bab 1.6 Mencegat Paket Jaringan (Netfilter Hooks) Hlm. 12

BAGIAN II: Arsitektur eBPF (The Kernel VM)

Bab 2.1 Dari Paket Filter Klasik ke eBPF Hlm. 14

Bab 2.2 Komponen BPF: Verifier & Safety First Hlm. 15

Bab 2.3 Just-In-Time (JIT) Compiler & Native Speed Hlm. 16

Bab 2.4 BPF Maps (Berbagi State dengan User-Space) Hlm. 17

Bab 2.5 Kprobes & Kretprobes (Dynamic Tracing) Hlm. 18

Bab 2.6 Tracepoints (Static Kernel API) Hlm. 19

Bab 2.7 eXpress Data Path (XDP) untuk Packet Drop Dini Hlm. 20

DAFTAR ISI (2/2)

BAGIAN III: Rekayasa Sensor (Sensor Engineering)

Bab 3.1 Pilihan Toolkit: BCC vs libbpf (C/Python) Hlm. 22

Bab 3.2 CO-RE (Compile Once – Run Everywhere) Hlm. 23

Bab 3.3 Kode: Menangkap Eksekusi dengan sys_execve Hlm. 24

Bab 3.4 Mengekstrak Argumen & String di Memory Hlm. 25

Bab 3.5 BPF Ring Buffer untuk Transmisi Data Asinkron Hlm. 26

Bab 3.6 Menulis User-Space Handler dengan Golang Hlm. 27

Bab 3.7 Batasan eBPF: Stack 512 Byte Hlm. 28

Bab 3.8 BPF Tail Calls (Melompati Batas Instruksi) Hlm. 29

BAGIAN IV: Mendeteksi & Menghancurkan Rootkits

Bab 4.1 Mengalahkan DKOM: Task Struct vs Scheduler Hlm. 31

Bab 4.2 Membongkar Port Siluman (TCP/UDP Unmasking) Hlm. 32

Bab 4.3 Mendeteksi Modifikasi pada Syscall Table Hlm. 33

Bab 4.4 Ancaman Baru: eBPF-based Rootkits (bpfkit) Hlm. 34

Bab 4.5 Fileless Malware & Eksekusi Memori (memfd) Hlm. 35

Bab 4.6 TOCTOU Mitigation di Ruang Kernel Hlm. 36

Bab 4.7 Kernel Panic (Oops) & Tracing Crash Logs Hlm. 37

Bab 4.8 Forensik Memori Melalui eBPF Hlm. 38

BAGIAN V: DevSecOps & Enterprise eBPF

Bab 5.1 Evolusi Enterprise: Isovalent Tetragon Hlm. 40

Bab 5.2 Aqua Tracee untuk Analitik Event Cepat Hlm. 41

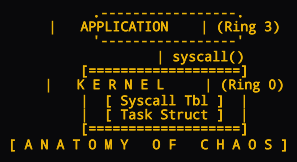
Bab 5.3 Cilium (Network Security & Kubernetes Identity) Hlm. 42

Bab 5.4 Integrasi Telemetri JSON eBPF ke SIEM Hlm. 43

Bab 5.5 Manajemen Overhead Performa (CPU/Memori) Hlm. 44

Bab 5.6 Kompatibilitas Versi Kernel Linux & Fallback Hlm. 45

Bab 5.7 Masa Depan Keamanan Kernel Cloud-Native Hlm. 46



BAGIAN I

Anatomi Kernel Linux
& Operasi Rootkits

BAB 1: MEDAN PERANG RING 0

1.1 Model Hak Istimewa (Ring 0 vs Ring 3)

Arsitektur prosesor x86 membagi hak istimewa eksekusi ke dalam lapisan-lapisan konsentris (*Rings*). Aplikasi normal (seperti peramban *Chrome*, peladen *Nginx*, dan bahkan alat deteksi antivirus) berjalan di **Ring 3 (User Space)**. Di lapisan ini, aplikasi dilarang keras menyentuh perangkat keras fisik (memori RAM mentah, kartu jaringan, atau disk) secara langsung.

Jantung sistem operasi (Kernel Linux/Windows) hidup di **Ring 0 (Kernel Space)**. Kernel adalah Tuhan atas sistem. Ia memiliki akses total ke segala hal.

Ketika *Nginx* ingin mengirim paket web ke jaringan, ia harus meminta izin kepada Kernel menggunakan **System Call (Syscall)**. Jembatan *Syscall* ini adalah pembatas suci. Jika peretas berhasil menginjeksi *Malware* ke dalam Ring 0 (menjadi *Rootkit Kernel*), konsep keamanan Ring 3 runtuh seketika, karena *Rootkit* kini memiliki wewenang untuk menipu dan membohongi semua aplikasi pengguna, termasuk *Security Agent* Anda.

1.2 Loadable Kernel Modules (LKM)

Pertanyaannya: Bagaimana peretas bisa memasukkan kodenya ke dalam Kernel Ring 0 yang diproteksi ketat? Vektor yang paling populer adalah menyalahgunakan fitur sah Linux bernama **Loadable Kernel Module (LKM)**.

LKM diciptakan oleh Torvalds agar pengguna bisa menambahkan *Driver* (seperti *driver printer* atau kartu grafis) tanpa perlu melakukan *Reboot* (*Restart*) pada sistem. Admin dapat memasang LKM dengan perintah `insmod` atau `modprobe`.

Peretas yang telah mendapatkan akses `root` melalui eksploitasi web (misalnya via *Tomcat/WordPress*) akan mengompilasi kode *Malware* mereka menjadi format modul kernel (`.ko`), lalu memuatnya menggunakan `insmod evil_rootkit.ko`. Begitu modul ini dimuat, ia menyatu dan menjadi bagian tak terpisahkan dari inti Kernel, berjalan dengan privilese *Ring 0* tertinggi, siap untuk membajak tabel instruksi sistem operasi utama.

1.3 Syscall Table Hooking

Setelah *Rootkit* masuk ke Kernel, tugas pertamanya adalah membajak (*Hooking*) **System Call Table** (Tabel fungsi utama OS).

Saat aplikasi memanggil `sys_read()` atau `sys_getdents()` (untuk membaca daftar direktori), Kernel merujuk ke tabel *Syscall* untuk menemukan alamat memori fungsi tersebut. *Rootkit* menimpa alamat fungsi asli di tabel ini dengan alamat fungsi palsu (*Hook*) miliknya sendiri.

```
// Mekanisme Logika Rootkit Syscall Hooking (C) // 1. Simpan fungsi asli agar kita bisa memanggilnya nanti original_sys_getdents = sys_call_table[__NR_getdents]; // 2. Fungsi Palsu kita (Hook) asmlinkage long hooked_sys_getdents(unsigned int fd, ... ) { // Panggil fungsi asli terlebih dahulu untuk dapatkan data aslinya long ret = original_sys_getdents(fd, ...); // MENGEDIT HASIL: Hapus file "evil_malware.elf" dari daftar kembalian! strip_hidden_file_from_buffer(); return ret; // Kembalikan data yang sudah dimanipulasi ke aplikasi (User) } // 3. Timpa tabel (Bypass proteksi Write) sys_call_table[__NR_getdents] = hooked_sys_getdents;
```

1.4 DKOM (Direct Kernel Object Manipulation)

Sistem operasi modern melindungi *Syscall Table* dengan menaruhnya di area memori *Read-Only*. Sebagai respons, *Rootkit* modern menggunakan **DKOM (Manipulasi Objek Kernel Langsung)**.



Di Linux, setiap proses disimpan dalam struktur `task_struct` dan diikat bersama dalam *Doubly Linked List* (Rantai memori). *Rootkit* DKOM tidak mengubah *Syscall*, ia **memanipulasi rantai memori ini secara langsung**. Ia memotong *Pointer Forward* dari `PID 100` agar langsung melompati `PID 666` dan menunjuk ke `PID 101`. Saat perintah `ps` atau fungsi kernel membaca rantai tersebut, proses 666 hilang dari daftar, menjadi hantu (*Ghost Process*) yang tak terlihat.

1.5 VFS & Menyembunyikan File

Selain menyembunyikan eksekusi (*Proses*), *Rootkit* harus menyembunyikan berkas konfigurasinya (*Malware Files*) dari pembacaan Disk (*Hard Drive*). Di Linux, segala sesuatu berujung pada **VFS (Virtual File System)**.

VFS adalah lapisan abstraksi yang memungkinkan Linux membaca disk format *ext4*, *xfs*, atau *FAT* menggunakan fungsi (API) yang sama persis. VFS menyimpan informasi fail dalam struktur yang disebut **Inode** dan operasi berkas di dalam `file_operations`.

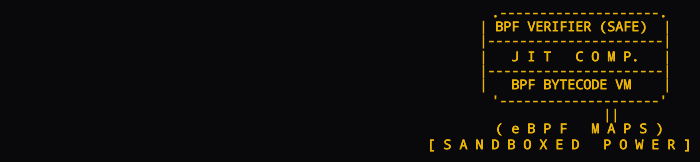
Eksploitasi *Rootkit* yang canggih (seperti *Diamorphine*) akan membajak fungsi operasi readdir (`iterate_shared` atau `filldir`) yang ada di struktur VFS. Ketika sistem mengiterasi/membaca seluruh berkas dalam folder `/tmp`, modul *Rootkit* akan membandingkan nama berkasnya. Jika namanya `evil_payload.bin`, fungsi manipulatif ini tidak akan memasukkannya ke dalam *Buffer* daftar kembalian. Fail secara fisik ada di Disk SSD Anda, tapi menghilang dari realitas sistem operasi.

1.6 Netfilter & Packet Hiding

Untuk menyembunyikan komunikasinya dengan *Server C2* (*Command and Control*), *Rootkit* tidak bisa sekadar menggunakan *Port* terbuka biasa, karena akan terdeteksi oleh `netstat` atau `ss`. Mereka menginjeksi **Netfilter Hooks**.

Netfilter adalah subsistem *Kernel Linux* (yang juga mendasari *iptables/ufw*). *Rootkit* mendaftarkan *hook* dengan prioritas tertinggi absolut (sebelum alat *Firewall* apa pun memprosesnya) di rantai `PREROUTING`.

Ketika paket jaringan (*Network Packet*) tiba di kartu jaringan (NIC), fungsi *hook Rootkit* memeriksanya. Jika paket tersebut mengandung *Magic String* tertentu (Misal: `"WAKEUP_MALWARE"` di *Header TCP*), *Rootkit* akan membacanya secara internal, lalu merespons dengan **NF_DROP**. Mengirim `NF_DROP` membuat seluruh sistem Linux (termasuk *Firewall* dan *Packet Sniffer* tcpdump di Ring 3) mengira paket tersebut tidak pernah ada, padahal instruksinya telah sukses dieksekusi oleh *Malware Kernel* secara senyap.



BAGIAN II

Arsitektur eBPF
(The Kernel Virtual Machine)

BAB 2: PARADIGMA EBPF

2.1 Dari Paket Filter ke Mesin Virtual

BPF awalnya diciptakan di awal 90-an (*Berkeley Packet Filter*) khusus untuk utilitas `tcpdump` agar filter jaringan (Mencari *Port 80*) berjalan cepat di Kernel tanpa perlu menyalin paket ke *User-Space* (yang memakan beban CPU).

Namun, pada 2014, Kernel Linux melakukan revolusi arsitektur: **extended BPF (eBPF)**. BPF bukan lagi sekadar filter jaringan; ia berevolusi menjadi **Mesin Virtual Turing-Complete di dalam Kernel**.

Alih-alih menulis Modul LKM C/C++ yang bisa menyebabkan *Kernel Panic*, *Developer* menulis kode sensor dalam format *Restricted C*. Kode ini dikompilasi menjadi *Bytecode* khusus eBPF. Kemudian, *Bytecode* ini disuntikkan ke dalam VM eBPF di Kernel. eBPF mengizinkan kita menyisipkan sensor (*Hook*) di HAMPIR SETIAP baris eksekusi kernel (Penjadwalan memori, *Syscall*, Jaringan) secara dinamis tanpa perlu melakukan *Reboot* atau mengubah *Source Code* OS sama sekali.

2.2 BPF Verifier (Safety First)

Alasan utama mengapa eBPF diizinkan berlari bebas di dalam *Ring 0* (yang sangat suci) tanpa menakut-nakuti Administrator adalah keberadaan komponen penjaga bernama **BPF Verifier (Pemverifikasi)**.

Sebelum *Bytecode* eBPF diizinkan masuk ke memori eksekusi Kernel, mesin *Verifier* akan menganalisis kode tersebut selama sepersekian detik secara matematis (*Static Analysis*).

Verifier memberlakukan hukum besi: Program eBPF **TIDAK BOLEH** memiliki iterasi (*Loop*) tak terbatas yang bisa membuat sistem macet (*Hang*). Program eBPF tidak boleh membaca alamat memori acak (*Dereference Null Pointer*) yang menyebabkan *Crash*. Program dibatasi jumlah maksimum instruksinya (awalnya 4096, kini 1 Juta). Jika kode sensor Anda gagal dalam uji keselamatan matematis ini, Kernel akan langsung menolak pemuatannya (*Load Failed*). Jaminan 100% *Crash-Free* ini menjadikan eBPF senjata utama bagi *Cloud* raksasa (AWS/GCP).

2.3 Just-In-Time (JIT) Compiler

Beroperasi sebagai Mesin Virtual (VM) terdengar identik dengan *Lambat (Overhead Tinggi)*, seperti Java JVM lama. Namun sensor eBPF beroperasi pada kecepatan absolut tingkat biner murni (*Native Speed*). Rahasianya adalah **JIT Compiler (Kompiler Seketika)**.

Setelah *Verifier* memastikan kode eBPF aman, kode *Bytecode* tersebut tidak dieksekusi oleh mesin virtual penerjemah. Modul JIT (di dalam Kernel) akan mengambil *Bytecode eBPF generik* tersebut dan menerjemahkannya secara langsung (Seketika/*Just-In-Time*) menjadi **Instruksi Bahasa Rakitan (Assembly) Mesin Fisik Asli (x86_64 atau ARM64)**.

Hasilnya, ketika sensor *Syscall* keamanan (eBPF) Anda aktif untuk mencegat proses tebas (*Malware execution*), eksekusi pencegatan tersebut tidak memakan tambahan beban (*Zero Overhead*) abstraksi, berjalan persis secepat fungsi Kernel asli yang dikompilasi oleh arsitek OS.

2.4 BPF Maps (Berbagi State)

Sensor eBPF yang hidup di dalam Kernel (Ring 0) terisolasi. Jika sensor menangkap IP peretas, bagaimana ia memberitahu aplikasi analis Python/Go Anda yang berada di *User-Space (Ring 3)*? Mekanisme penghubungnya adalah **BPF Maps**.

BPF Maps adalah struktur data penyimpanan (Seperti *Hash Table, Array, LRU Cache*) berkinerja luar biasa tinggi yang dialokasikan di dalam memori Kernel, namun **dapat dibaca dan ditulis secara asinkron dari kedua sisi (Kernel dan User-Space)**.

Skenario Pertahanan: Aplikasi *Python User-Space* Anda mengunduh daftar IP *Threat Intel* (100.000 IP Jahat) dan memuatnya ke dalam *BPF Hash Map*. Sensor eBPF (di Ring 0) menyadap lalu lintas masuk; pada setiap paket, ia mengecek IP Asal melawan *Hash Map* ini (Berjalan dalam kecepatan nano-detik). Jika *Match* (Cocok), eBPF langsung membuang paket tersebut (*Packet Drop*) tanpa pernah membangunkan CPU untuk meneruskan paket ke *User-Space*, menghemat daya komputasi secara masif.

2.5 Kprobes & Kretprobes

Di mana saja (Titik Tembel) kita bisa menempelkan sensor eBPF ini? Alat paling dinamis adalah **Kernel Probes (Kprobes)**.

Dengan Kprobes, *Security Engineer* dapat menyisipkan sensor ke hampir setiap inci baris fungsi internal kernel secara dinamis. Jika Anda ingin menyadap setiap pembukaan file, pasang *Kprobe* di fungsi `vfs_open()`.

Saat fungsi sasaran dipanggil, CPU mengenai *Breakpoint* dinamis (Int 3), eksekusi dijeda, lalu melompat mengeksekusi program eBPF Anda (Menginspeksi argumen register: *Siapa yang memanggil fungsi ini? File apa yang dibuka?*).
Selain Kprobe, ada juga **Kretprobe (Kernel Return Probe)**. Kretprobe disisipkan tepat saat fungsi tersebut *selesai* bekerja dan mengembalikan nilai. Ini krusial untuk melacak kode *Error* (*Apakah file berhasil dibuka atau Permission Denied?*).

2.6 Tracepoints (Static API)

Kelemahan utama dari *Kprobes* adalah **Ketidakstabilan API (API Instability)**. Fungsi internal Kernel Linux (seperti `tcp_v4_connect`) bisa berubah nama argumen atau parameternya pada setiap peluncuran versi Kernel Ubuntu baru. Sensor eBPF *Kprobe* Anda yang berjalan sempurna di Ubuntu 20.04 bisa rusak dan *Error* di Ubuntu 22.04.

Solusi yang lebih kuat (*Robust*) untuk perangkat lunak korporat adalah **Tracepoints**.

Tracepoints adalah titik tembel statis yang sengaja disediakan secara keras (*Hardcoded*) oleh pengembang Kernel Linux untuk melacak *Event* penting (Seperti `sys_enter_execve` untuk memantau eksekusi program baru). Struktur data dan nama argumen dari *Tracepoints* dijamin secara arsitektur (*ABI Stable*) tidak akan pernah berubah lintas versi OS (Dari kernel lawas hingga modern). Sebisa mungkin, Arsitek eBPF selalu memprioritaskan *Tracepoints* sebelum turun tangan menggunakan *Kprobes*.

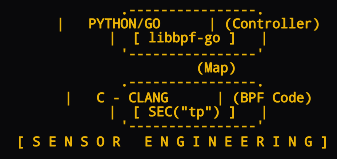
2.7 eXpress Data Path (XDP)

Untuk melumpuhkan Serangan Volumetrik (DDoS Jaringan Raksasa), bahkan *iptables/Netfilter* masih terlalu lambat karena paket jaringan telanjur diproses menjadi struktur data `sk_buff` oleh OS.



XDP (eXpress Data Path) adalah tipe program eBPF ekstrem yang disuntikkan LANGSUNG ke dalam sirkuit *Driver* Kartu Jaringan (NIC) di luar jangkauan tumpukan sistem operasi. Ia mencegat paket mentah (*Raw Bits*) saat baru saja keluar dari kabel jaringan fisik.

Dengan kode XDP, eBPF dapat merespons paket serangan DDoS dengan perintah mutlak **XDP_DROP** dalam hitungan nanodetik. *Server* yang sebelumnya sekarat di 1 Juta Paket/Detik kini mampu menelan 20 Juta Paket/Detik (*Wire-Speed Drop*) tanpa satupun CPU kernel OS berkeringat, mendefinisikan ulang batas Anti-DDoS generasi awan modern.



BAGIAN III

Rekayasa Sensor
(Sensor Engineering C & Py)

BAB 3: PEMROGRAMAN EBPF

3.1 BCC vs libbpf (Masa Lalu vs Modern)

Mengembangkan sensor eBPF membutuhkan penggabungan dua bahasa: Anda menulis kode Sensor (yang berjalan di Kernel) menggunakan *Restricted C*, lalu Anda menulis aplikasi Pengendali (yang membaca hasil peringatan) di *User-Space* menggunakan Python, Go, atau Rust.

Di era awal (2015-2019), alat standar (*Toolkit*) yang digunakan adalah **BCC (BPF Compiler Collection)**. BCC menggabungkan (*Inline*) kode C Anda ke dalam skrip Python. Masalah fatalnya: BCC harus melakukan proses kompilasi *Clang/LLVM* secara *Real-Time* (*Runtime Compilation*) di setiap *Server Target Produksi* Anda. Ini mengharuskan terpasangnya paket raksasa *Kernel Headers* dan *Compiler Clang* (ratusan Megabyte) pada peladen web *Nginx* operasi Anda. (Mimpi buruk bagi Container Docker yang mungil).

Arsitektur modern beralih ke **libbpf** murni. Anda mengkompilasi silang (*Cross-compile*) kode C menjadi biner `.o` sekali saja di laptop pengembang Anda, lalu mengirim biner *Bytecode* kecil (ukuran 20 KB) tersebut ke semua kontainer/*Server* operasi tanpa perlu memasang *Compiler/Headers* lagi di lingkungan Produksi.

3.2 CO-RE (Compile Once – Run Everywhere)

Tantangan terbesar libbpf sebelumnya adalah **Offset Memori (Memory Layout)**. Di Kernel Linux versi 5.4, struktur data `task_struct` mungkin menyimpan nilai `PID` di *offset* (jarak memori) *Byte ke-500*. Namun, di Kernel 5.15, nilai `PID` bergeser ke *Byte ke-512*.

Jika program eBPF membaca secara membabi-buta di *Offset* 500, program itu akan membaca sampah (*Garbage*) pada Kernel 5.15 dan membunyikan alarm *False Positive*. Inilah alasan alat lawas harus di-*recompile* terus-menerus per versi kernel.

Revolusi akhirnya datang dalam bentuk teknologi **BTF (BPF Type Format) & CO-RE**. Kernel Linux modern menyertakan *Metadata BTF* bawaan (peta ukuran presisi) dari dirinya sendiri (disimpan tersembunyi di `/sys/kernel/btf/vmlinux`). Saat sensor *Bytecode* CO-RE eBPF Anda dimuat ke versi Kernel asing, sistem libbpf secara otomatis (*Relocation*) membaca peta BTF ini, menyesuaikan *Offset Memory* (seperti mengkalibrasi teropong tembak) secara dinamis. Anda mengkompilasi sekali di *Laptop* Anda, berjalan aman di ribuan *Server Enterprise* dengan 100 versi Kernel yang berbeda.

3.3 Kode: Menangkap Eksekusi Proses

Kasus penggunaan keamanan (EDR) paling fundamental adalah menangkap peringatan setiap kali proses baru (*Malware/Shell*) dijalankan. Kita akan menggunakan *Tracepoint* `sys_enter_execve`.

```
// Sensor Kernel (C / libbpf) - sensor.bpf.c #include <vmlinux.h> #include <bpf/bpf_helpers.h> //
Definisi Struct Data untuk dikirim ke User-Space struct event_t { u32 pid; u32 uid; char comm[16]; }; //
Definisi Ring Buffer Map struct { __uint(type, BPF_MAP_TYPE_RINGBUF); __uint(max_entries, 256 * 1024); }
events SEC(".maps"); // Tracepoint Macro pada Syscall Execve (Jalankan Program)
SEC("tracepoint/syscalls/sys_enter_execve") int trace_execve(struct trace_event_raw_sys_enter *ctx) {
struct event_t *e; e = bpf_ringbuf_reserve(&events, sizeof(*e), 0); // Pesan tiket buffer if (!e) return
0; e->pid = bpf_get_current_pid_tgid() >> 32; // Dapatkan Process ID e->uid = bpf_get_current_uid_gid();
// Dapatkan User ID bpf_get_current_comm(&e->comm, sizeof(e->comm)); // Nama File Induk
bpf_ringbuf_submit(e, 0); // Tembakkan (Submit) Ke Python/Golang! return 0; } char LICENSE[]
SEC("license") = "GPL";
```

3.4 Mengekstrak Argumen (Memori Kernel)

Mendapatkan *Process ID (PID)* itu mudah, namun mengekstrak Argumen Command Line (Misalnya teks `"-c wget http://malware"` saat bash dijalankan) jauh lebih menantang. Di eBPF, Anda dilarang membaca memori secara sembarangan.

Teks Argumen dari `execve(filename, argv, envp)` terletak di Ruang Pengguna (*User-Space Memory*). Sensor eBPF berjalan di Kernel. Jika eBPF membaca (*Dereference*) *pointer* `argv` tersebut langsung, mesin *Verifier* akan menendang Anda (*Safety Violation*) karena memori User-Space bisa berubah (terpental ke swap disk/*Page Fault*) kapan saja.

Satu-satunya cara sah adalah menggunakan fungsi bantu (*Helper Function*) khusus: `bpf_probe_read_user_str()`. Fungsi bawaan ini bertindak sebagai asisten pembaca yang aman. Ia menyalin senyap teks *String* (seperti argumen *Malware*) dari memori *User-Space*, menjamin tidak akan ada *Kernel Crash/Page Fault*. Teknik pembacaan pelindung (*Defensive Read*) ini adalah rahasia 100% kestabilan alat *Enterprise*.

3.5 BPF Ring Buffer (Transmisi Data Cepat)

Terdapat dua struktur data pengiriman pesan (*Events*) dari Kernel ke Aplikasi Pengendali SIEM: **Perf Buffer (Lama)** dan **BPF Ring Buffer (Baru)**.

Di masa lalu, *Perf Buffer* memaksa alokasi pita memori (*Memory Bandwidth*) tersendiri untuk *SETIAP* inti CPU (*Per-CPU Buffer*). Jika *Server Data Center* Anda memiliki 128 Inti CPU, Anda akan merusak RAM dengan alokasi 128 antrean terpisah. Lebih buruknya, ini menyebabkan masalah (*Out-of-Order Events*) ketika pesan log EDR CPU ke-2 tiba di SIEM lebih cepat daripada CPU ke-1.

BPF Ring Buffer (Kernel 5.8+) menyelesaikan krisis tersebut dengan pendekatan Antrean Memori Bersama yang terkunci (*Shared Lockless-Queue*). Semua 128 CPU berbagi satu Ring Buffer tunggal secara sinkron dan efisien. Ini tidak hanya menghemat alokasi RAM korporat hingga 90%, tetapi juga menjamin bahwa Log Alarm keamanan (*Execution Events*) akan selalu dibaca (*Polled*) oleh aplikasi SIEM dalam urutan waktu Kronologis mutlak (*Sequential Ordering*).

3.6 User-Space Handler (Golang)

Sensor kernel (C) Anda hanyalah pelempar data. Tangkapan tersebut diproses (*Parsed*) dan diteruskan ke SIEM/Kafka oleh **Pengendali Ruang Pengguna (User-Space Handler)**. Industri keamanan awan (*Cloud-Native/Kubernetes*) telah memilih **Golang** (pustaka `cilium/ebpf` atau `libbpfgo`) sebagai bahasa standar emas (bukan Python) karena performa konkurensi (*Goroutines*) dan kompilasi binari statis (*Single Binary Deployment*).

Aplikasi *Go* Anda memuat (*Load*) fail `sensor.o` (Biner eBPF) ke Kernel melalui fungsi *Syscall BPF*. Setelah terpasang, aplikasi *Go* masuk ke putaran (*Polling Loop*) tak terbatas, menempel ke *Ring Buffer Descriptor (FD)*. Setiap kali eBPF (di Kernel) menembakkan tiket `bpf_ringbuf_submit`, aplikasi *Go* secara instan menerimanya, mengubah data *Byte* mentah (C-Struct) tersebut menjadi format *JSON Log* standar perusahaan (misal OCSF), dan memuntahkannya ke REST API Elasticsearch / Kafka.

3.7 Batasan Absolut eBPF: Stack 512 Byte

Insinyur C/C++ biasa sering frustrasi saat pindah ke eBPF. *Verifier* sangat kaku dan memberlakukan batas keras (*Hard Limit*): **Ukuran total memori Stack program eBPF HANYALAH 512 Bytes.**

Sebagai perbandingan, program *User-Space* (seperti Nginx) memiliki *Stack* sebesar 8 Megabyte. Jika di eBPF Anda mengetik `char buffer[1024];` untuk menampung *String Path* direktori *Rootkit* panjang, saat Anda mencoba kompilasi (*Clang*) dan *Load*, *Verifier* akan langsung memuntahkan (*Error: Stack out of bounds*) tanpa ampun.

Solusi Arsitek untuk menyalin Argumen besar/String Direktori tanpa menggunakan memori *Stack* adalah **Per-CPU Array Maps**. Alih-alih membuat variabel *array* lokal di *Stack* (C-Code), insinyur mendeklarasikan *BPF Map* global dengan tipe BPF_MAP_TYPE_PERCPU_ARRAY berukuran 32 Kilobita. Mesin eBPF mengambil *Pointer Storage* besar dari *Map Heap* tersebut saat dibutuhkan, dan melepaskannya kembali. Kebebasan komputasi melalui teknik peminjaman memori *Stateful*.

3.8 BPF Tail Calls (Melompati Batas)

Sebelum kernel modern, program eBPF tunggal juga dibatasi hanya 4096 instruksi (kini 1 Juta). Namun, menjejalkan seluruh mesin pendeteksi *Malware (Network, File, Process)* ke dalam satu berkas monolitik (*Monolithic C-File*) akan menciptakan kode (*Spaghetti*) yang sangat lambat divalidasi *Verifier* (bisa memakan waktu 3 menit proses muat / *Load Time*). Solusi pemecahannya adalah **Tail Calls (Pemanggilan Ekor)**.

Tail Calls mengizinkan Program eBPF A melompat ke Program eBPF B secara langsung (*Chaining Execution*). Lompatan ini bukanlah panggilan fungsi (*Function Call*) biasa yang menambah ketebalan *Stack Frame*. Saat Program A melompat (`bpf_tail_call`), eksekusi A dihancurkan seketika, dan *Frame* digantikan seutuhnya oleh Program B (seperti perintah `execve`), menghindari ledakan memori (*Stack Overflow*).

Arsitektur *Enterprise EDR* memecah sensor menjadi modul mikro (1 Program Router/Pusat, 1 Program Parser DNS, 1 Program Deteksi Ransomware File). *Tail Calls* bertindak bagai persimpangan jalan rel kereta, menavigasi paket log ke gerbong spesialisnya dalam orde *Zero-Overhead Jump* (hitungan instruksi Assembli murni).



BAGIAN IV

Mendeteksi & Menghancurkan
Rootkits Jaringan

BAB 4: MEMBURU HANTU RING 0

4.1 Mengalahkan DKOM (Cross-Referencing)

Kita telah membahas bahwa *Rootkit* DKOM memotong (*Unlink*) `task_struct` (Proses *Malware*) dari daftar, membuatnya hilang dari perintah `ps`. Ini adalah manipulasi Daftar (*Linked List*) Pasif.

Meskipun hilang dari daftar, proses *Malware* **TETAP HARUS DIJALANKAN OLEH CPU** agar program penambang Kripto-nya dapat bekerja. Penjadwal CPU Kernel (*CPU Scheduler* / `sched_entity`) memiliki daftar *Runqueue* (Antrean Lari) terpisah di memori intinya. Menghapus proses dari *Runqueue* ini setara dengan menghentikan (membunuh/*Kill*) proses tersebut secara harfiah. Karenanya, *Rootkit* **TIDAK PERNAH** berani menghapus dirinya dari jadwal *Runqueue Scheduler*.

Deteksi eBPF: Kita menempelkan sensor *Kprobe* langsung di fungsi jatung detak (*Heartbeat Scheduler*) Kernel (misal: `finish_task_switch`). eBPF akan mencatat setiap PID yang fisiknya benar-benar dieksekusi CPU milidetik demi milidetik. *User-Space Handler* kita kemudian membandingkan (*Cross-Reference*) daftar ini dengan daftar dari API `/proc/` standar (yang sudah ditipu Rootkit). Jika PID 666 aktif di CPU (eBPF *Log*), namun tidak ada di `/proc/` (Log OS), alarm KRITIKAL (**Hidden Process Detected**) dibunyikan.

4.2 Membongkar Port Siluman (TCP Unmasking)

Taktik kedua Rootkit LKM adalah menyembunyikan soket jaringan (*Network Socket*) C2 di Port 1337 dari pantauan alat pembaca status (*Netstat/ss*). Rootkit melakukannya dengan mem-*bypass / hook* fungsi operasi pembacaan *Inodes TCP* di dalam subsistem /proc/net/tcp.

Deteksi eBPF: Sensor pertahanan kita tidak mengandalkan fungsi pembaca (Status). Kita memantau interaksi **Aktif (State Change)**. Kita menempatkan *Tracepoints eBPF* pada fungsi inti `inet_csk_accept` (untuk Panggilan Masuk/*Bind Listener*) dan `tcp_v4_connect` (untuk Panggilan Keluar).

Setiap kali proses (termasuk hantu PID 666) mencoba membuka jembatan *TCP Handshake* baru, fungsi level-bawah *TCP Connect* (mesin statis OS yang tak bisa dibohongi tanpa merusak seluruh protokol internet) akan tereksekusi. Sensor eBPF mencatat *Socket Tuple* (IP Asal, IP Tujuan, dan *Port* terlarang). Sekali lagi, perbedaan daftar *Socket Hash Table* Kernel dengan balasan statis *netstat* yang dicurangi, seketika membongkar tempat persembunyian hantu tersebut.

4.3 Mendeteksi Modifikasi Syscall Table

Mencegah Rootkit lawas yang membajak (*Hook*) alamat `sys_call_table` sebenarnya cukup lugas (*Straightforward*). Namun mengeksekusi inspeksi ini dalam waktu nyata (*Runtime Realtime*) 100 kali sedetik melalui modul deteksi (LKM EDR klasik) akan membebani CPU secara destruktif.

Pendekatan eBPF menggunakan pemeriksaan Timer Berkala (*eBPF Timer/Perf Events*) atau Memicu *Hook* pada setiap modifikasi Memori Laman Inti. Skrip BPF menyimpan Peta/*Map Hash Baseline* berisi daftar 300+ rentang Alamat Memori (*Memory Range Pointers*) Fungsi Kernel asli yang sah (diambil dari Peta BTF (*BPF Type Format*) atau `/boot/System.map` OS saat *Booting* murni).

Sensor eBPF (Ring 0) melakukan pembacaan memori (*dereferencing pointer*) `sys_call_table[sys_read]` setiap beberapa detik. Jika alamat *Pointer* tersebut menunjuk **DI LUAR BATAS (Out of Bounds)** wilayah teks Kernel OS (`.text`), dan menunjuk ke area memori asing (Daerah *Module LKM Malware*), mesin Deteksi Anomali Pointer *eBPF Kernel* langsung menyimpulkan tabel utama telah ditimpa (*Syscall Table Hooked*).

4.4 Ancaman Baru: eBPF-based Rootkits

Setiap pisau memiliki dua sisi. Teknologi pengawasan tanpa jejak (*Zero Overhead, Stealth*) yang ditawarkan eBPF juga memikat hati para pengembang *Malware* (Black Hats). Lahirlah ancaman generasi modern: **eBPF Rootkits (seperti bpfkit atau BoopKit)**.

Rootkit eBPF JAUH lebih berbahaya daripada Rootkit LKM. Jika *Rootkit* LKM (`.ko`) bisa dideteksi dengan mudah via perintah `lsmod` (*List Modules*), *Rootkit eBPF* tidak muncul di sana (karena ia bukan LKM). Malware eBPF menginjeksi *Bytecode XDP* atau *Tracepoints* ke dalam mesin Virtual Kernel, menggunakan hak akses administrator (Root/CAP_BPF), dan memanfaatkan Map Peta (*BPF Maps*) sebagai *Key-Value Store* Command & Control (C2) mereka.

Mereka dapat membajak argumen fungsi `sys_read`. Saat *User* membaca fail, *Rootkit eBPF* menimpa kepingan *Buffer* memori data balasan (*Return Data Overwrite* menggunakan fungsi BPF) seketika sebelum data itu mencapai layar pengguna. Ilusi yang disokong (Disponsori) langsung oleh subsistem sah Kernel sendiri.

4.5 Fileless Malware & Eksekusi Memori

Evolusi penghindaran EDR (*Evasion EDR*) berikutnya adalah serangan tanpa file (*Fileless Malware*). *Malware* tradisional menaruh berkas biner `.elf` di Disk SSD yang mudah dipindai oleh Antivirus Kriptografi (seperti YARA Scanner). *Malware* Siluman menggunakan trik **memfd_create()**.

Syscall memfd_create memungkinkan proses membuat berkas (*File*) Anonim yang HIDUP MURNI DI DALAM RAM (*Memory-Backed*), tak pernah menyentuh pelat magnetik (Disk) fisik sama sekali.

Peretas mengunduh *Malware* via skrip, menaruhnya di dalam fail RAM `memfd` tersembunyi (Biasanya hanya muncul sebagai struktur *inode* rahasia), lalu mengeksekusinya via instruksi `fexecve`. Antivirus konvensional akan lumpuh total karena tidak ada aktivitas I/O *Hardisk* yang memicu alarm pemindaian.

Mitigasi eBPF: EDR Cloud (Tracee/Tetragon) menyematkan *Kprobe* spesifik yang menjebak `memfd_create` yang digandengkan dengan eksekusi turunan (`exec`). Jika silsilah keturunan ini terekam bersamaan (*Parent-Child Correlation*), eBPF akan memblokir *Syscall*-nya dan menyedot isi biner memori anonim tersebut untuk diumpankan ke Analisis Forensik Dinamis.

4.6 TOCTOU Mitigation di Ruang Kernel

Salah satu alasan mengapa EDR lawas (berbasis *User-Space*) gampang dikalahkan adalah kerentanan Waktu (**Time-of-Check to Time-of-Use / TOCTOU**). Saat EDR meminta Kernel membaca (*Check*) alamat memori Argumen Eksekusi *Malware*, Argumen itu mungkin terlihat normal (Misal: `/bin/echo Hello`). EDR (User) menyetujuinya.

Namun, dalam durasi sepersekian milidetik jeda *Network Loop* aplikasi, *Thread Malware* lain dalam jaringan menyerang dan menukar argumen memori tersebut di RAM secara paralel (menjadi: `/bin/sh -c M4Lw4re`). Saat Kernel (*Use*) akhirnya mengeksekusinya, *Malware* jahatlah yang berjalan, bukan *echo*. Alat keamanan tertipu manuver rotasi penukaran kilat.

Arsitektur Pencegahan (LSM BPF): eBPF *Linux Security Modules* terbaru mengait ke dalam API Kernel *Enforcement* internal (Mekanisme Pencegatan Blokir). *LSM Hooks* menjeda eksekusi tepat *di dalam memori pembekuan (Frozen Atom)* sirkuit Kernel. Tidak ada benang tambahan (*Threads*) yang diizinkan untuk menyentuh memori tersebut hingga sensor eBPF menjatuhkan vonis Mutlak `ALLOW/DENY`, menghapuskan celah jeda *Race Condition TOCTOU* dari eksistensinya.

4.7 Tracing Kernel Panics (Oops Logs)

Saat *Threat Hunter* berhadapan dengan *Malware Zero-Day* yang sangat canggih (atau pengembang *Malware* amatir yang membuat *Coding C Error*), sistem Linux korporat Anda mungkin tiba-tiba meledak (*Crash / Freeze*). Kernel menyebut momen kematian fatal ini sebagai **Kernel Panic** atau **Oops**.

Menganalisis Forensik sistem *Panic* (karena layar hanya menjadi hitam teks *Dump* register *Hexadecimal*) sangat menakutkan (*Intimidating*). Analis elit (*Level 3*) menggunakan alat **kdump/crash** dan Sensor Penanda eBPF.

Sensor eBPF kita secara berkelanjutan menyisipkan catatan *Breadcrumbs* (Remah Roti) ke dalam Peta *Buffer Map Memory Ring-0* setiap kali ada Modul LKM tak dikenal dimuat, atau terjadi deviasi *Syscall Table*. Karena memori ini adalah bagian memori Ring-0 yang dilindungi saat *Crash/Dump*, berkas `vmcore` (Sisa Darah Memori yang dikumpulkan pasca-Restart) akan memuat catatan murni jejak detik terakhir *Malware* tersebut menginjeksi *Payload* (Rute Tumpukan Kode/*Stack Trace*), mengarahkan Analis tepat ke Alamat Offset Pemicu Bencana di dalam biner *Malware* pelaku.

4.8 Forensik Memori Melalui eBPF

Aktivitas respons insiden siber (DFIR) masa lalu mewajibkan Analis menancapkan *Flashdisk USB Tool* khusus (seperti LiME) untuk secara fisik menduplikat seluruh kapasitas 32GB RAM *Server* yang terinfeksi (*Full Memory Dump*), lalu memindahkannya lewat kabel (Penyalinan berjam-jam) ke Lab untuk diperiksa oleh *Framework Volatility*.

Di Lingkungan Awan Terdistribusi (Kubernetes) berkapasitas ribuan Pod kontainer skala masif, membekukan *Server* untuk melakukan *Full Dump RAM* adalah hal yang sangat purba dan memakan biaya pemadaman tak terbatas (*Downtime*).

Arsitektur DFIR (Forensik Dinamis) kini diserahkan kepada agen mikroskopis eBPF (seperti *bpf-forensics*). Alih-alih menyalin 32GB RAM buta, sensor eBPF ditugaskan *menyelinap masuk* ke Kernel yang hidup (*Live Target*), membaca struktur *Tree Node* rahasia khusus yang ditarget (Misal: *File Descriptors Tersembunyi* atau *Kunci Dekripsi TLS Soket* milik Container), dan mentransmisikan data temuan forensik presisi bedah ukuran Kilobita tersebut (*Surgical Memory Forensics*) kembali ke SIEM tanpa *Server Database* (*Pod Target*) sadar atau jeda pelayanan seinci pun.



BAGIAN V

DevSecOps & Enterprise eBPF
(Tetragon, Tracee, Cilium)

BAB 5: SKALA ENTERPRISE

5.1 Evolusi Enterprise: Isovalent Tetragon

Menulis kode C eBPF manual (*Custom Scripts*) sangat menyenangkan bagi peneliti. Namun, mendeploy ribuan skrip buatan sendiri yang tak teruji ke 10.000 peladen AWS Produksi (Kubernetes) Perusahaan akan mengundang bencana pemadaman operasional.

Industri beralih ke agen eBPF skala korporat (*Open Source Enterprise*). Yang menduduki takhta tertinggi (Diakuisisi oleh Cisco) adalah **Tetragon (dari Isovalent)**.

Tetragon memelopori arsitektur **Smart In-Kernel Filtering**. Sebagian besar EDR lama menyalin (*Export*) semua *Event Execve* secara membabi-buta ke *User-Space (Python Agent)*, membebani RAM (Overhead >20%). Tetragon mengeksekusi saringan 100% di dalam Ring-0. Anda memberikan aturan YAML: *"Hanya laporkan kejadian jika berkas /etc/shadow disentuh."* Tetragon mengompilasi logika Filter YAML ini menjadi Bytecode C, menyuntikkannya, dan Kernel akan membuang 99.9% *Events* tidak relevan (*Silent Drop*). Hanya 1 *Event Malware* (*True Hit*) yang diteruskan, menghasilkan performa gahar (Overhead <2%).

5.2 Aqua Tracee untuk Analitik Cepat

Alat raksasa *Open Source* pesaingnya adalah **Tracee (Aqua Security)**. Tracee dirancang khusus untuk memonitor kontainer *Kubernetes/Docker*.

Masalah terbesar Analis SOC dengan log sistem Kernel biasa (misal dari perintah `strace/auditd`) adalah kehilangan **Konteks Awan (Cloud Context)**. Jika Kernel melaporkan *"PID 4545 menjalankan bash"*, analis kebingungan: PID 4545 milik mesin siapa? Milik Pod *Frontend* Aplikasi atau Pod *Database* Terisolir?

Tracee secara cerdas menghubungkan (*Enrichment*) C-Struct rendah dari API Kernel dengan API Kubernetes Master. Saat Tracee memuntahkan *Alert JSON* ke SIEM, ia menyisipkan *Metadata Kubernetes Label (PodName, Namespace, ContainerID)*. Analis melihat: *"PID 4545 (Pod: Web-Nginx-Prod) menjalankan bash"*. Tracee juga dimuat dengan puluhan aturan deteksi tanda tangan *Malware OOTB (Out-of-The-Box)* berbasis OPA/Rego (Menangkap *Credential Theft* atau *Container Escape* harian) menjadikan pemasangan pelindung SOC secara utuh tanpa menulis baris kode sedikit pun (*Zero Code Policy*).

5.3 Cilium (Network Security & Kubernetes Identity)

Selain EDR untuk eksekusi proses (Tetragon/Tracee), pengawasan eBPF juga mendominasi area Jaringan: **Cilium (Cloud Native Computing Foundation)**.

Tumpukan jaringan Kubernetes standar (Kube-Proxy berbasis *iptables*) sangat menyedihkan secara performa jika jumlah layanannya (*Services*) melampaui 5000 Pod, karena *iptables* membaca aturan filter IP secara linear (*O(N)*). Membutuhkan detik yang sangat lama hanya untuk memuat rute koneksi antar-Aplikasi.

Cilium membuang *iptables* 100% dan menggantinya dengan peta *Hash* berkecepatan milidetik (*O(1) Map Hash*) murni eBPF (Bypass OS Net Stack). Untuk Keamanan (Security/CISO), Cilium memegang kunci emas **L7 Network Policy berdasarkan Identitas Aplikasi**. Cilium mengetahui bahwa *Kontainer Nginx* memiliki Identitas API (*Label X*) dan dilarang untuk melakukan koneksi POST HTTP ke *Kontainer Pembayaran* (Label Y). Jika koneksi dilanggar (oleh *Malware RCE*), Cilium membunuh/mencekiknya seketika (*TCP Reset*) dan memancarkan log forensik visualisasi ke *Hubble UI* Dasbor Operasional Jaringan.

5.4 Integrasi Telemetri JSON ke SIEM

Sensor Kernel (Tetragon/Tracee/Cilium) harus disalurkan ke otak pengambil keputusan korporat. Proses penyaluran *Data Pipa Awan* diatur melalui kerangka **Fluent-Bit atau Vector**.

Output dari Tetragon adalah garis log berkas JSON standar panjang yang dicetak (*Printed*) terus menerus ke saluran */var/log/tetragon/tetragon.log*. Aplikasi proksi pengirim log kilat (Seperti *Vector/FluentD*) memanen berkas JSON tersebut, menambah filter (Misalnya: Membuang log kejadian Kubernetes yang normal agar menghemat biaya lisensi Splunk), lalu memompa *Streaming Ingest* hasil filterisasi bersih (Anomali *Rootkit/Execve/Syscall*) tersebut melalui sambungan Broker **Apache Kafka**.

Dari Kafka, miliaran data *Endpoint Security* mendarat di *Splunk (SIEM)* atau *Elastic Security (Lakehouse)*. Dasbor SIEM yang ditatap para Analis L1 akan merender *Alert Peringatan Tinggi* seketika (*Real-time*), menjadikan integrasi *Ring 0 Sensor* dengan *Ring 7 Analysts* paripurna (Tersinkronisasi *End-to-End Enterprise SecOps*).

5.5 Manajemen Overhead Performa

CISO dan CTO selalu berbenturan saat pengimplementasian agen (*Agent EDR*) keamanan di *Server* Produksi: *"Jangan mematikan bisnis kami demi keamanan. Agen Anda memakan 30% CPU Database kami!"* keluh Pengembang Produk.

Ini adalah titik penjualan (*Selling Point*) magis utama eBPF: **Performa Tak Terlihat (Invisible Footprint)**.

Arsitek harus merekayasa agen *Tracee/Tetragon* mereka secara presisi. Kesalahan pemula adalah menyalakan (*Subscribe*) fungsi pemantauan pembacaan berkas (`vfs_read`) di semua direktori. *Server Database* (PostgreSQL/MySQL) melakukan JUTAAN kali bacaan berkas sedetik. Menyadap ini akan mencekik (*Overload*) CPU (Overhead >40%).

Security Engineer (DevSecOps) mengatur (*Tuning*) deklarasi *Filter Ring 0* (di sisi Kernel, bukan *User*): *"Hanya jebak operasi vfs_read JIKA lokasi direktorinya adalah folder sistem (/etc/, /bin/) ATAU jika pemanggil operasinya (Current Executable) beroperasi dari dalam folder penampung file sementara (/tmp/). Abaikan seluruh folder `/var/lib/postgresql`."* (CPU Overhead menjadi 0.5% Murni).

5.6 Kompatibilitas Versi Kernel (Fallback)

Meskipun kemajuan teknologi *BTF CO-RE* sangat ajaib, teknologi modern Kernel (seperti eBPF Ring Buffer) baru diluncurkan dan hanya berfungsi secara matang di atas Kernel Linux Versi 5.8+.

Lanskap korporat (Data Center Bank atau Rumah Sakit) adalah ekosistem warisan kaku (*Legacy System*). Mereka masih menjalankan *CentOS 7 (Kernel 3.10)* atau *Ubuntu 18.04 (Kernel 4.15)* untuk menopang *Server* Mesin Vital mereka. Anda tidak bisa mendeploy agen sensor eBPF terbaru di atasnya, *Verifier* kuno tersebut (atau Ketiadaan *Verifier/BTF* murni) akan langsung menolak pemasangan kompilasi Anda.

Skrip EDR Korporat (*Golang User Handler*) dirancang dengan **Fallback Architecture (Mekanisme Penurunan)**. Skrip akan mencium (*Probe Detection*) versi Kernel *Target Server*. Jika kernel modern (Versi 5.8+) eBPF CO-RE Sensor diturunkan ke dalamnya (Operasi kecepatan cahaya). Jika Skrip mengenali Kernel Kuno (Versi 3/4), sistem Penurunan (Fallback) terpicu otomatis (*Graceful Downgrade*) dan menanamkan LKM/Modul C kuno lama (atau API *Auditd* Kaku tradisional) yang boros daya namun tetap menjaga ketiadaan Zona Buta (*Zero Blind-Spots*).

5.7 Masa Depan Keamanan Kernel Cloud-Native

Selama dekade (2010 - 2020), para aktor siluman peretas (*Rootkit LKM / APT Malware*) memiliki keuntungan operasional asimetris. Mereka bermain, bersembunyi, dan mengobrak-abrik Jantung (Ring 0) Sistem Operasi Linux Korporat Anda, sementara para ahli keamanan bertahan dengan tameng plastik usang buta di lapisan luar (Ring 3 *Antivirus Scan / Splunk Regex Analytics*).

Kelahiran sirkuit maya eBPF mengubah kurva taktis medan perang ini selamanya. Mesin BPF mengizinkan Pembela Keamanan/Insinyur Awan (*The Blue Teams/SOC*) untuk turun ke dalam ring gladiator yang sama persis (Ring 0) di arena yang dikuasai peretas, bahkan dengan tingkat pengawasan kecepatan cahaya dan otorisasi visibilitas yang jauh lebih aman (melalui perlindungan *BPF Verifier*) dibandingkan *Rootkit Malware* ilegal yang kotor dan tidak stabil tersebut.

Insinyur Sistem (DevSecOps) yang fasih dan menguasai mantra *C / eBPF Bytecode*, akan selalu memegang kendali penglihatan Mahatahu (*Omniscient Observability*) dari petabita klaster armada *Cloud-Native Server* perusahaan di abad ini. Mereka tidak lagi dapat dibutakan oleh hantu ilusi manipulasi sistem manapun.

REFERENSI & DOKUMENTASI RISET

- **[1]** Gregg, B. (2019). *BPF Performance Tools: Linux System and Application Observability*. Addison-Wesley Professional. (Referensi Alkitab Sistem Observasi).
- **[2]** The Linux Kernel Organization. (2024). *BPF Documentation & bpf-helpers API API*. Linux Kernel Archives (kernel.org).
- **[3]** Isovalent (Cilium Project). (2021). *eBPF - Introduction, Tutorials & Community Resources* (ebpf.io).
- **[4]** Blunden, B. (2012). *The Rootkit Arsenal: Escape and Evasion in the Dark Corners of the System (2nd Edition)*. Jones & Bartlett Learning. (Arsitektur Manipulasi Kernel/DKOM Tradisional).

GLOSARIUM KERNEL & EBPF (A-L)

TERMINOLOGI	DEFINISI & ARSITEKTUR
CO-RE (BTF)	*Compile Once - Run Everywhere*. Mekanisme eBPF libbpf (Membaca Meta Peta tipe C / BTF) yang memfasilitasi pelacak (*sensor*) berkalibrasi-sendiri untuk menyesuaikan tata letak memori struktur (*struct offsets*) dari berbagai OS Linux (*Kernel Version*) tanpa *recompile*.
DKOM	*Direct Kernel Object Manipulation*. Vektor peretasan *Malware Rootkit* canggih. Menghindari pendeteksian pengaitan tabel *Syscall*, DKOM langsung memotong pita kaitan-kaitan (*Linked-list Pointer*) RAM dalam OS. Menghilangkan proses PID jadi Hantu Mutlak.
eBPF Verifier	Otak keselamatan absolut eBPF VM Kernel. Pemverifikasi statik yang membaca kode Anda sebelum dijalankan. Jika ia menemukan Loop tak terhingga (*Infinite*) atau Potensi *Memory Crash*, ia langsung menendang Biner alat (*Tool*) Anda dari Kernel (*Drop/Fail*).
LKM (Rootkits)	*Loadable Kernel Module (Biner .ko)*. Ekstensi suci penambah fitra (*Driver HW*) ke dalam inti Linux. Sering disalahgunakan para APT (Advanced Persistent Threats) menyisipkan *Malware* Level-Tuhan di *Ring 0 (Rootkit LKM)* OS Anda.

GLOSARIUM KERNEL & EBPF (M-Z)

TERMINOLOGI	DEFINISI & ARSITEKTUR
Ring Buffer (BPF)	Sirkuit peta memori khusus yang digunakan untuk memuntahkan temuan log Deteksi (C-Struct biner) dari *Sensor Kernel eBPF Ring 0* menembak naik secara sinkronisasi terkunci efisien ke arah *Aplikasi Pengendali Go/Python User-Space Ring 3*.
Syscall (Table)	*System Call / Tabel Panggilan Sistem*. Fungsi Suci Gerbang API batas wilayah Pengguna OS & Inti Sistem. (Misalnya: *Read, Write, Execve*). Sasaran empuk pembajakan (*Hooking*) Rootkits kuno.
Tracepoints / Kprobes	Titik-titik sisipan tempelan *Sensor/Probe eBPF*. **Kprobe** bersifat liar dan menempel ke mana saja (Fungsi apa pun, Rawan *Crash OS Update*). Sedangkan **Tracepoints** diukir dan dilindungi stabilitas format parameter resminya (*API Stable*) oleh Arsitek pembuat *Kernel*.
XDP	*eXpress Data Path*. Modul eBPF jaringan ekstrem. Bertugas memblokir paket serangan jaringan (*DDoS Drops*) sedini mungkin di level gerbang Sinyal Perangkat Keras Kabel (*NIC Driver Level*), sebelum paket itu menguras CPU perute OS sedikitpun.



Ketika peretas menembus benteng terakhir dan mengendalikan ilusi sistem operasi Anda, satu-satunya cara membongkar kebohongan tersebut adalah dengan turun menjadi penguasa absolut di medan pertempuran yang sama: Ring 0 (Kernel).

The Ring 0 Guardians

© 2026 All rights reserved.
Enterprise eBPF Sensor Architectures